

Charotar University of Science and Technology [CHARUSAT]

Chandubhai S. Patel Institute of Technology [CSPIT]

U & P U Patel Department of Computer Engineering

Date: 12/12/2025

Practical List

Academic Year	:	2025-26	Semester	:	4
Course Code	:	CSUE203	Course Name	:	FOUNDATION OF DATA SCIENCE AND ANALYTICS

Sr. No.	Aim	Hrs.	CO
1.	Pandas & NumPy Basics – Series and Data Frame Operations	2	1
	Objective: To apply NumPy and Pandas for creating and manipulating one-dimensional (Series) and two-dimensional (DataFrame) data structures using real-world tabular data. Setup and Imports - Install and verify installation of NumPy and Pandas in the existing virtual environment. - Import NumPy as <code>np</code> and Pandas as <code>pd</code> in a Python script or Jupyter Notebook. - Check and print the versions of NumPy and Pandas being used. NumPy Array and Pandas Series - Create one-dimensional NumPy arrays from Python lists (integers, floats, and mixed types where appropriate). - Perform basic NumPy operations: element-wise addition, subtraction, multiplication, division, and aggregation (mean, max, min, standard deviation). - Convert NumPy arrays into Pandas Series and assign custom indexes (e.g., day names, student IDs). - Access Series elements using: - Integer positions (<code>s[0]</code>, <code>s[1:4]</code>) - Label-based indices (<code>s['Mon']</code>, <code>s['Mon':'Thu']</code>) - Use common Series methods such as <code>head()</code>, <code>tail()</code>, <code>value_counts()</code>, and <code>describe()</code>. Pandas DataFrame Basics		

- Create a Pandas **DataFrame** from:
 - A dictionary of Python lists.
 - A NumPy 2D array.
- Load data from a CSV file (e.g., `nyc_weather.csv` or `weather_data.csv` from the provided folder).
- Display the first and last few rows of the DataFrame using `head()` and `tail()`.
- Inspect the structure of the DataFrame using `shape`, `columns`, `index`, `dtypes`, and `info()`.
- Use `describe()` to compute basic statistics of numeric columns.

Data Selection and Basic Analysis

- Select single and multiple columns using: `df['col']` and `df[['col1', 'col2']]`.
- Select rows using:
 - `loc` with labels (e.g., `df.loc[2]`, `df.loc[2:5]`)
 - `iloc` with integer positions (e.g., `df.iloc[0:3, 1:3]`)
- Filter rows based on conditions (e.g., temperatures > certain value, sales above a threshold).
- Use NumPy functions (`np.mean`, `np.max`, `np.min`) on DataFrame columns and compare with Pandas equivalents (`df['col'].mean()`, etc.).
- Add new calculated columns using vectorized operations (e.g., °C → °F, total marks from individual subject columns).

Saving and Exporting Data

- Save selected columns or filtered data into a new CSV file using `to_csv()` (with and without index).
- Verify the saved file by reloading it and checking the data.

Assignment for the Students

1. Create a NumPy array representing daily sales for 7 days and convert it into a Pandas Series with day names as indices.
2. Using a CSV file containing student marks (or `indata.csv`), create a DataFrame and:
 - Display basic statistics (min, max, mean, standard deviation) using both NumPy and Pandas.
 - Add a new column for “Percentage” using a NumPy-based formula.
3. Write a program that loads a CSV file, selects only numeric columns, and computes:
 - Column-wise mean and median using NumPy.
 - The row with the highest total value across selected columns.

	4. Save the cleaned and augmented DataFrame to a new CSV file and verify the number of rows and columns after saving.		
2.	Data Cleaning with Pandas and NumPy – Handling Missing and Inconsistent Data	2	1
	Objective: To handle missing, inconsistent, and noisy data in tabular datasets using Pandas functions (<code>isnull</code>, <code>dropna</code>, <code>fillna</code>, <code>replace</code>) and NumPy statistics. Loading and Inspecting Data with Missing Values - Load a CSV/Excel file with missing values (e.g., <code>weather_data.csv</code>, <code>missing_data.txt</code>) into a DataFrame. - Identify missing values using: <code>isnull()</code> / <code>isna()</code> and <code>notnull()</code> <code>df.isnull().sum()</code> to count missing values per column. - Display rows where specific columns have missing values (e.g., rows where <code>temperature</code> is missing). Removing Missing Data - Use <code>dropna()</code> to: - Remove rows with any missing values. - Remove rows where all values are missing. - Remove rows only if specific columns have missing values using <code>subset</code> parameter. - Compare the shape of the DataFrame before and after applying <code>dropna()</code>. Filling Missing Data - Replace missing values using <code>fillna()</code> with: - A constant (e.g., 0, “Unknown”). - The mean/median of the column computed using NumPy (<code>np.mean</code>, <code>np.median</code>) and Pandas (<code>df['col'].mean()</code>). - Forward fill (<code>method='ffill'</code>) and backward fill (<code>method='bfill'</code>) strategies. - Demonstrate filling missing dates or temperature values to create a continuous time series. Replacing Inconsistent Values		



- Use `replace()` to standardize inconsistent categorical values, such as:
 - `'Yes', 'Y', 1 → 'Yes'`
 - `'No', 'N', 0 → 'No'`
- Replace out-of-range numeric values (e.g., negative temperatures or unrealistic marks) with `NaN` and then handle them using `fillna()` and NumPy-based statistics.

Grouping and Aggregation with Cleaned Data

- Group data using `groupby()` on one or more categorical columns (e.g., `city`, `department`).
- Compute aggregated statistics (mean, count, max, min, standard deviation) using both Pandas aggregations and NumPy functions inside `agg()`.
- Plot or print summary tables showing per-group statistics (e.g., average temperature per city, average marks per class).

Saving Cleaned Data

- Save the cleaned and processed DataFrame to a new CSV/Excel file (`cleaned_weather.csv`, etc.).
- Verify that missing values are handled appropriately and that no inconsistent labels remain.

Assignment for the Students

1. Using a dataset with missing values (weather, sales, or student marks):
 - Count how many missing values exist per column.
 - Replace missing numeric values with the column mean using NumPy.
2. Create a DataFrame that intentionally contains:
 - Missing values
 - Inconsistent labels (e.g., `'M'`, `'Male'`, `'male'`)
 Clean this DataFrame using `fillna()`, `replace()`, and demonstrate the “before” and “after” outputs.
3. Perform a `groupby()` operation on cleaned data (e.g., by city or class) and compute mean, max, and count using NumPy functions inside `agg()`.
4. Save the cleaned and grouped summary to a new file and write a short comment (in a code cell or comments) on how data cleaning changed the results compared to the original raw dataset.

3.	Advanced Pandas – Concatenation, Merge/Join, Pivot, Melt, Stack/Unstack,	2	1
-----------	---	----------	----------

	and Crosstab		
	Objective: To perform advanced data reshaping and relational operations using Pandas (<code>concat</code>, <code>merge</code>, <code>pivot</code>, <code>melt</code>, <code>stack</code>, <code>unstack</code>, <code>crosstab</code>) and integrate NumPy-based computations into the analysis. Concatenation and Appending Data • Create multiple small DataFrames (e.g., monthly sales or partial student lists) using dictionaries, lists, or NumPy arrays. • Use <code>pd.concat()</code> to: ◦ Concatenate DataFrames vertically (add rows). ◦ Concatenate DataFrames horizontally (add columns) with <code>axis=1</code>. • Reset indices and handle duplicate indexes using <code>ignore_index=True</code>. Merging and Joining Data • Prepare two or more DataFrames representing related entities (e.g., students, marks, courses) using CSV files like <code>stock_data.csv</code>, <code>stock_data.xlsx</code>, or custom examples. • Use <code>merge()</code> to perform different joins: ◦ Inner join ◦ Left join ◦ Right join ◦ Outer join • Demonstrate merges based on: ◦ Common column(s) ◦ Index vs. column using <code>left_index</code> and <code>right_index</code>. • Use NumPy operations on merged DataFrames (e.g., compute total cost = <code>quantity * price</code> using vectorized multiplication). Pivot, Pivot Table, and Melt • Use <code>pivot()</code> to reshape DataFrame from long format to wide format (e.g., dates as rows, cities as columns, temperature as values using <code>weather2.csv</code> or <code>weather3.csv</code>). • Use <code>pivot_table()</code> with: ◦ Aggregation functions (<code>mean</code>, <code>max</code>, <code>len</code>)		

- Multiple index and columns
- Fill missing values in pivot tables using `fill_value` or NumPy operations.
- Use `melt()` to convert wide-format data back into long-format, specifying `id_vars` and `value_vars`.

Stack, Unstack, and Crosstab

- Convert hierarchical column structure into a MultiIndex and use `stack()` and `unstack()` to reshape data between rows and columns.
- Illustrate how `stack()` converts columns into row-level index and how `unstack()` reverses it.
- Use `pd.crosstab()` to create frequency tables between two categorical variables (e.g., city vs. weather type, department vs. grade).
- Integrate NumPy operations (e.g., normalizing counts using NumPy division to get percentages).

Mini Case Study: Sales / Attendance / Weather Analysis

- Use two or more CSV files (e.g., products, orders, customers OR weather for different cities/months).
- Perform the following:
 - Concatenate monthly data into a single DataFrame.
 - Merge with a lookup table (e.g., product prices or city information).
 - Create a pivot table summarizing key metrics (e.g., total sales per month per product, or average temperature per city per month).
 - Generate a crosstab to count occurrences (e.g., number of orders by payment type and customer segment).
 - Use NumPy functions to compute additional derived metrics (e.g., percentage contribution of each product/city).

Assignment for the Students

1. Prepare three small CSV files (or DataFrames) representing quarterly sales and:
 - Concatenate them into a single annual sales DataFrame.
 - Merge with a product price list to compute total revenue per product using NumPy-based calculations.

	- Create a pivot table that shows total or average values per category (e.g., department vs. semester, city vs. month). Use <code>pivot_table()</code> with appropriate aggregation functions and fill missing entries. - Use <code>melt()</code> to convert a wide-format DataFrame (e.g., marks in multiple subjects as separate columns) into a long format with columns: Student, Subject, Marks. - Generate a <code>crosstab()</code> between two categorical fields in your dataset and then convert counts into percentage distribution using NumPy operations. - Write a short reflection (as comments or markdown) explaining where Pandas was more convenient than pure NumPy for tabular data, and where NumPy's operations were still useful inside the Pandas workflow.		
4.	Exploratory Data Analysis (EDA) of Air Quality Data in India	2	2
	You are tasked with performing Exploratory Data Analysis (EDA) on the Air Quality Data in India using Jupyter Notebook or Google Colab. This dataset contains measurements of various pollutants (PM2.5, PM10, NO2, SO2, CO, O3) across multiple Indian cities and dates. Your objective is to explore the dataset by calculating descriptive statistics to understand central tendencies and dispersion, identifying and handling missing values, analyzing correlations between pollutants using a correlation matrix, and generating visualizations such as line plots, boxplots, histograms, and heatmaps to reveal trends, anomalies, and relationships in the data. The goal is to develop a meaningful understanding of pollution patterns across time and location while gaining hands-on experience in basic EDA techniques. Key Questions / Analysis / Interpretation to be Evaluated - Interpret spatial and temporal pollution patterns. - Analyze interdependence among pollutants using correlation matrices. - Evaluate data quality and justify handling strategies for missing values. Supplementary Problems Compare pollution levels during winter and summer months across the top 5 polluted cities using visual analysis. Key Skills Addressed Data Cleaning and Pre-processing, Descriptive Statistical Analysis, Correlation Analysis, Data Visualization Learning Outcome		

	By the end of this practical, learners will: 1. Perform Comprehensive EDA: Explore datasets using descriptive statistics (e.g., mean, median, standard deviation). Visualize trends, anomalies, and relationships using appropriate plots. 2. Handle Data Quality Issues: Identify missing data and apply strategies like imputation or deletion. 3. Analyze Correlations: Interpret pollutant relationships using correlation matrices and heatmaps. 4. Interpret Trends: Use visual analysis to compare seasonal pollution patterns across cities. Dataset Source: https://www.kaggle.com/datasets/rohanrao/air-quality-data-in-india/data Description: Context Air is what keeps humans alive. Monitoring it and understanding its quality is of immense importance to our well-being. Content The dataset contains air quality data and AQI (Air Quality Index) at hourly and daily level of various stations across multiple cities in India. Cities Ahmedabad, Aizawl, Amaravati, Amritsar, Bengaluru, Bhopal, Brajrajnagar, Chandigarh, Chennai, Coimbatore, Delhi, Ernakulam, Gurugram, Guwahati, Hyderabad, Jaipur, Jorapokhar, Kochi, Kolkata, Lucknow, Mumbai, Patna, Shillong, Talcher, Thiruvananthapuram, Visakhapatnam. Tools/Technology Python, pandas, scikit-learn, matplotlib, seaborn		
5.	Comprehensive House Price Analysis Project	2	2
	Objective: To perform a full exploratory data analysis (EDA) on a housing dataset to identify key drivers of house prices and build a basic predictive model. Syllabus Topics Covered: Data Cleaning (missing values, outliers), Descriptive Statistics, Univariate & Bivariate Analysis, Feature Selection (correlation), Regression Analysis (Simple and Multiple). Task: Data Loading & Initial Cleaning: Load a dataset (e.g., Ames Housing		

or Melbourne Housing) and handle **missing values** (e.g., imputation or removal).

Descriptive & Univariate Analysis: Calculate all primary statistical measures (mean, median, range, std dev). Visualize the distribution of the target variable (SalePrice) using **histograms** and **box plots** to understand its shape and identify **outliers**.

Bivariate Analysis & Feature Selection: Visualize relationships between potential features and the price using **scatter plots** and a **correlation heatmap**. Select the top 3-5 most correlated features.

Modeling (Simple & Multiple Regression): Build a simple linear regression model using only the *single best* predictor. Then, build a multiple linear regression model using all 3-5 selected features. Compare the performance (e.g., R-squared) of the two models.

Key Questions / Analysis / Interpretation to be Evaluated

Data Quality: Evaluate the completeness of the dataset and justify the chosen strategies for handling missing values and identified outliers.

Feature Importance: Analyze and interpret which specific house features (e.g., area, number of rooms) show the strongest positive or negative correlation with the final sale price.

Model Performance: Compare the performance of the single-variable linear regression model versus the multiple-variable linear regression model.

Learning Outcome

By the end of this practical, learners will:

Perform Comprehensive EDA: Explore a real-world tabular dataset using descriptive statistics, data cleaning techniques, and appropriate visualizations.

Handle Data Quality Issues: Identify and apply strategies (imputation, deletion) to manage missing and inconsistent data points effectively.

Analyze Correlations: Interpret variable relationships using correlation matrices and heatmaps to inform feature selection decisions.

Apply Regression Basics: Implement and evaluate simple and multiple linear regression models to predict a continuous target variable.

Dataset Link: <https://www.kaggle.com/datasets/dansbecker/melbourne-housing-snapshot>

Tools/Technology

Python, pandas, scikit-learn, matplotlib, seaborn

6.	Employee Satisfaction Study with Advanced Hypothesis Testing	2	2
	Objective: To use rigorous hypothesis testing to determine if factors like department, location, or tenure have a significant impact on employee satisfaction or performance scores. Syllabus Topics Covered: Data Sampling, Hypothesis Testing, ANOVA (One-way and Two-way), Outlier Detection. Task: Data Curation & Sampling: Find or generate a dataset containing employee data (scores, location, department, etc.). Implement a specific sampling technique, such as stratified random sampling, to ensure representative subsets for analysis. Outlier Analysis: Identify and visualize outliers in performance scores using methods like Z-scores or box plots. One-Way ANOVA: Formulate a hypothesis to test if satisfaction scores are significantly different across various office locations. Run the ANOVA test in Python and interpret the p-value. Two-Way ANOVA (Advanced): Investigate if both department and location (and their interaction) affect the scores. Perform a Two-Way ANOVA analysis and summarize your findings. Key Questions / Analysis / Interpretation to be Evaluated Sampling Strategy: Justify the choice of stratified sampling and explain how it helps ensure a representative analysis of employee data. Hypothesis Interpretation: Interpret the results of the ANOVA tests (p-value, F-statistic) to determine if significant differences in mean satisfaction scores exist across various departments and locations. Specific Group Differences: Analyze the post-hoc test results (Tukey's HSD) to identify exactly which specific pairs of groups (e.g., Department A vs. Department C) exhibit statistically different mean scores. Learning Outcome Apply Data Sampling: Implement effective data sampling techniques to prepare representative datasets for statistical analysis. Perform Hypothesis Testing: Formulate and test statistical hypotheses about population means using One-Way and Two-Way ANOVA tests.		

	Interpret Statistical Results: Analyze and interpret the outputs of complex statistical tests (ANOVA, post-hoc tests) to draw data-driven conclusions about group differences. Handle Data Quality Issues: Identify and manage outliers within the dataset prior to analysis. Dataset Link: https://www.kaggle.com/datasets/raminhuseyn/hr-analytics-dataset		
7.	Multivariate & High-Dimensional Data Visualisation	2	3
	Objectives: • To understand principles of effective visualization and chart selection strategies • To visualize multivariate relationships in data • To explore high-dimensional data patterns using t-SNE • To interpret insights from complex datasets Dataset (UCI Repository): Wine Quality Dataset ☐ https://archive.ics.uci.edu/ml/datasets/wine+quality (Contains physicochemical attributes → suitable for multivariate & high-dimensional visualization) Tools: Python, Pandas, Matplotlib, Seaborn, Scikit-learn Tasks: 1. Load and preprocess the Wine Quality dataset 2. Apply Principles of Effective Visualization (clarity, simplicity, accuracy) 3. Perform the following visualizations: ○ Scatter Plot Matrix (pairwise feature relationships) ○ Correlation Heatmap ○ Bubble Plot (feature vs feature with quality as size) ○ Parallel Coordinate Plot (multi-feature comparison) 4. Apply t-SNE for high-dimensional visualization: ○ Reduce dimensions to 2D ○ Visualize clusters based on wine quality		

	5. Analyze patterns and draw conclusions about feature impact on quality Expected Learning Outcome: ● Ability to select appropriate charts for multivariate data ● Hands-on experience with high-dimensional visualization techniques ● Improved analytical interpretation of complex datasets		
8.	Time Series, Geospatial & Interactive Data Visualization	2	3
	Objectives: ● To visualize temporal trends using time-series charts ● To represent spatial data using geospatial visualizations ● To create interactive dashboards ● To communicate insights effectively through data storytelling Datasets (UCI Repository) 1. Air Quality Dataset (Time Series) https://archive.ics.uci.edu/ml/datasets/air+quality 2. World Population Dataset (for Geospatial Mapping) https://archive.ics.uci.edu/ml/datasets/world+population Tools: Python, Pandas, Matplotlib, Plotly, Google Looker Studio / Tableau Tasks: 1. Load and clean time-series data 2. Create Time Series Visualizations: ○ Line Chart (trend analysis) ○ Candlestick Chart (variation analysis) 3. Perform Geospatial Visualizations: ○ Choropleth Map (population / pollution levels) ○ Dot Density Map ○ Hexagonal Binning Map 4. Build an Interactive Dashboard using: ○ Google Looker Studio or Tableau ○ Filters, tooltips, and real-time interaction 5. Apply Storytelling with Data: ○ Define problem statement ○ Create narrative flow (context → insight → conclusion) ○ Present dashboard insights Expected Learning Outcome:		

	• Ability to visualize time-dependent and spatial data • Hands-on experience with interactive dashboards • Skill to communicate insights effectively using storytelling techniques		
--	--	--	--

Prepared By:	Prof. Mrugendrasinh Rahevar, Dr. Amit Thakkar, Prof. Dharmendrasinh Rathod, Prof. Mikin Patel, Prof. Hemant Yadav and Prof. Deep Kothadiya
---------------------	--